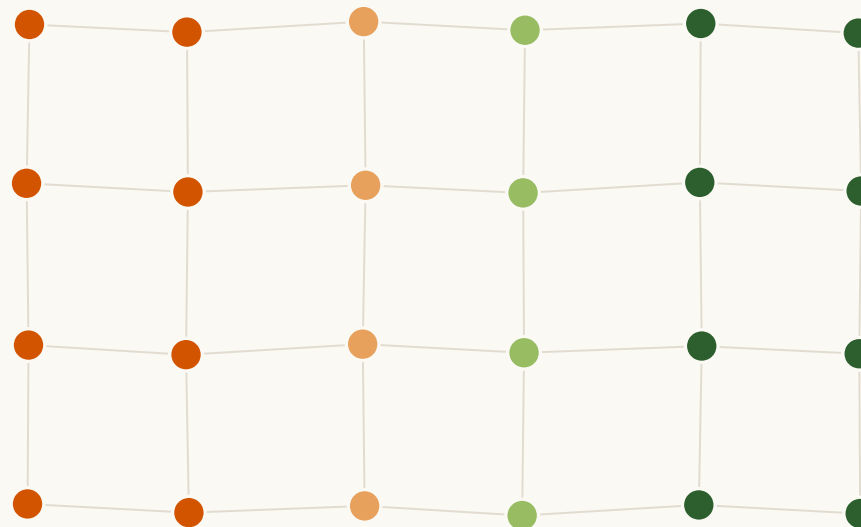


제 24 회 임베디드 SW 경진대회 · 자유공모 부문

불사

화중생련 火中生蓮

고장 (노드 파괴) 을 데이터로 바꾸는 자가치유 산불 감시 메시



재난에선 노드 파괴가 '필연' 이다

산불 · 지진 · 붕괴 현장에서 센서 노드는 **반드시 죽는다** . 기존 시스템은 이 죽음을 단순 '손실' 로 처리한다 . 우리는 **죽음의 시공간 패턴 자체를 데이터로 승격**해 , 사라진 노드들이 오히려 불의 방향 · 속도를 그리게 한다 .

문제 정의

위성 · 드론은 연기 · 수목 아래 지표 화선을 못 본다 . 진화 지휘 · 대원 안전이 직결된 실시간 공백이 남는다 .

기존의 공백

stock 메시는 죽은 노드를 우회만 할 뿐 , 그 죽음에 담긴 정보를 버린다 .

우리 목표

고장 패턴을 데이터로 + 자가치유로 , 그 공백을 지상에서 실시간으로 채운다 .

→ 응용 (산불) 이 아니라 아키텍처 (고장→데이터 + 자가치유) 에 독창성이 있다 .



상공이 못 보는 ' 지표 화선 ' 의 실시간 공백

무엇이 안 보이냐

연기와 수목 캐노피 아래에서 실제로 번지는 지표 화선 (fire front) 의 위치 · 방향 · 속도 . 위성은 광역 개요만 , 드론은 상공 · 간헐적이라 초 단위 국지 변화를 놓친다 .

왜 치명적인가

진화 지휘부의 자원 배치와 최전선 대원의 대피 타이밍이 바로 이 정보에 달려 있다 . 공백이 곧 인명 · 재산 피해로 이어진다 .

현장 동기 (2025 의성 산불)

대형 산불 대응 현장에서 , 지상의 실시간 화선 정보가 없어 판단이 지연되는 국면을 보며 ' 지상에서 화선을 실측하는 저가 레이어 ' 의 필요를 절감했다 . (동기 · 삽화로만 사용 — 반사실 주장 아님)



고장을 '손실'이 아니라 '데이터'로

① 노드 파괴

불에 타 죽는 순간

② 죽은 시각 · 좌표

= 불이 도착한 시각

③ 방향 · 속도 · ETA

불의 진행을 역산

목표 1 — 고장의 데이터화

교차검증된 '죽음'을 확정 이진 신호로 모아, 흩어진 (좌표 · 죽은시각)에서 화선의 위치 · 방향 · 속도를 추정한다.

목표 2 — 자가치유

노드가 죽어 경로가 끊겨도 남은 노드들이 자동으로 우회로를 다시 짜, 관측 자체가 계속되게 한다.

→ 두 목표가 합쳐져: 노드는 죽지만(불에 타도) 시스템의 관측은 '죽지 않는다'. = 불사(不死)의 이중 의미.



하드웨어 · 소프트웨어 구성

하드웨어

- 센서 노드 : ESP32 × 16 (WiFi 메시)
- 온도 감지 : DS18B20 디지털 온도센서
- 상태 표시 : WS2812 LED (생사 · 경보 시각화)
- 게이트웨이 : 라즈베리파이 (메시↔대시보드)
- 전원 : 배터리 / 승압 모듈

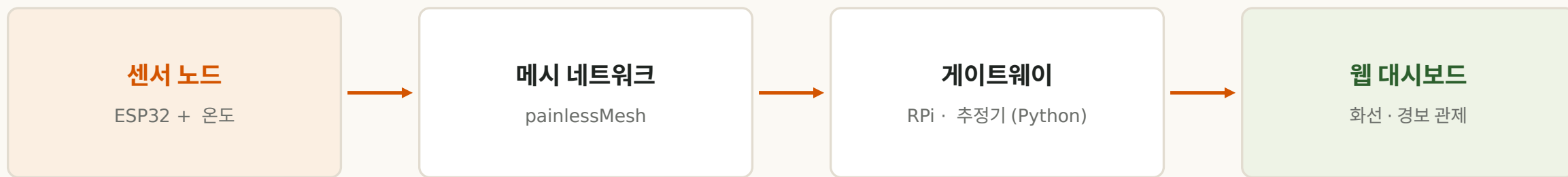
16 노드 실물 재료비 ≈ 45 만 원 (저가 소모형 설계).

소프트웨어

- 노드 펌웨어 : C/C++ (Arduino-ESP32 + painlessMesh)
 - 메시 자동 구성 · 우회 + 메시 시각 동기 제공
- 게이트웨이 : Python — 시물 추정기 코드 그대로 재사용
- 관제 : 자체완결 웹 대시보드 (HTML, 서버 불필요)
- 개발 · 검증 : 네트워크 시뮬레이터 (Python), Wokwi(컴파일 확인)



데이터 흐름 아키텍처



노드가 임종신호 · 하트비트를 메시로 흘리면 , 게이트웨이가 죽은 노드의 시공간 패턴에서 화선을 추정하고 , 대시보드가 생사 · 자가치유 경로 · 참 / 추정 화선 · 대피경보를 실시간 재생한다 .

핵심 이식성 증명

게이트웨이가 시뮬레이터의 추정기 코드를 그대로 재사용 → 모의 데이터에서 방향 2.115° · 속도 0.096% 가 시물 직접값과 완전 일치 . 검증된 알고리즘을 실물 파이프라인에 그대로 옮길 수 있음을 증명 .

시스템을 이루는 네 개의 핵심 모듈

①

임종신호 (Last-Gasp)

노드가 임계온도를 넘겨 죽기 직전 쏘는 마지막 패킷 — 죽음의 시각을 확정.

②

자가치유 라우팅

노드가 죽어 길이 끊기면 sink 기준 역방향 BFS 로 우회로를 자동 재계산.

③

오탐 방어 (교차검증) ★

통신두절 vs 진짜 파괴를 다중이웃 + 온도로 구분 — stock 메시가 못 하는 계층.

④

도착시각 추정 ★

죽은 (좌표 · 시각) 에서 최소제곱으로 불의 방향 · 속도 · ETA 를 역산.



죽은 노드의 (좌표 · 시각) → 불의 방향 · 속도

1

죽은 시각 수집

각 노드가 임계온도에 타 죽은 시각 = 불이 그 지점에 도착한 시각 .

2

최소제곱 평면 적합

흩어진 (x, y, 죽은시각) 점들에 가장 잘 맞는 면을 긋는다 (딥러닝 아님).

3

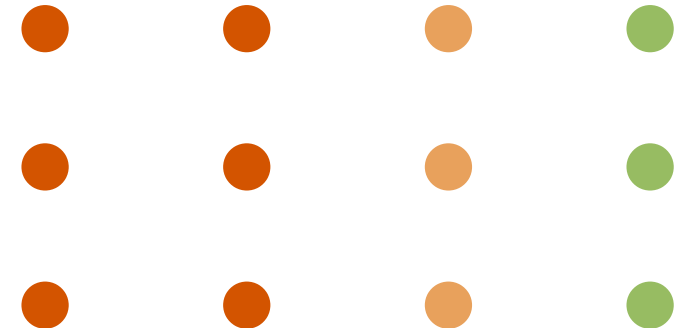
기울기 → 방향 · 속도

평면의 기울기 방향 = 진행 방향 , $1 / \text{기울기 크기} = \text{진행 속도}$.

도착시각 지도 (개념)

먼저 죽음

나중 죽음



불 진행 방향

가벼운 최소제곱 — 저사양 게이트웨이에서 즉시 계산 . **방향 견고 (2.1°)**, 속도는 보수적 (안전마진).



'통신두절' 인가, '진짜 파괴' 인가

노드가 조용해졌다고 다 죽은 게 아니다. 잠깐 끊긴 것 (통신두절) 을 죽음으로 오판하면 가짜 화선이 그려진다. 우리는 두 층으로 구분한다.

1 층 · 다중 이웃 교차검증

한 노드가 침묵해도 즉시 죽음으로 보지 않는다. K 개 이웃 ($K_{confirm}=3$) 이 독립적으로 '연결 끊김' 을 확인해야 사망 후보로 승격 — 일시적 링크 노이즈를 걸러낸다.

2 층 · 온도 교차검증

사망 후보가 직전에 고온을 겪었는가? 화선에 의한 죽음이면 온도 상승이 선행한다. 온도 정황이 뒷받침될 때만 '화선 사망' 으로 확정.

결과 : 측정 전 구간 (통신두절 0.30 까지) 오탐률 0%. 초기 설계의 관대판정發 오탐 (0.20~) 을 원인 특정해 제거. 이 계층이 죽음을 단순 우회하는 stock 메시와의 진짜 차별점이다.



길이 끊겨도 관측은 죽지 않는다

자가치유 라우팅

- sink(id 0) 기준 역방향 BFS 로 각 노드의 상행 경로를 구성 .
- 노드가 DEAD 로 확정되면 영향 구역만 경로를 즉시 재계산 .
- 연결성이 유지되는 한 , 죽은 노드를 우회해 데이터가 계속 sink 에 도달 .

※ 재라우팅 실시연은 실물에서 첫 실측 (Phase C). 시물의 100ms 는 틱값 (성능 아님).

임종 신호 (Last-Gasp)

- 노드가 임계온도를 넘어 죽기 직전 , 마지막 패킷을 방출 .
- ' 언제 죽었는가 ' 를 확정 → 도착시각 추정의 입력 시각을 정밀화 .
- 하트비트 (주기 신호) 누락과 결합해 침묵을 판정 .

죽음의 ' 시각 ' 이 정확할수록 화선 방향 · 속도 추정이 정밀해진다 .

자가치유 (관측 지속) + 임종 신호 (죽음의 시각 확정) 가 만나 , ' 죽는 노드 ' 가 ' 살아있는 정보 ' 가 된다 .



코드 구성과 'measure-first' 규율

저장소 구성

- sim/ — 네트워크 시뮬레이터 · 도착시각 추정기
- firmware/ — ESP32 노드 펌웨어 (C/C++)
- gateway/ — 라즈베리파이 추정기 (Python, sim 재사용)
- dashboard/ — 자체완결 웹 관제
- docs/ — 연구노트 (PROGRESS·DECISIONS·STRESS_REPORT)

결정 로그 (D-001~) · 진행 로그를 남겨 '정직한 연구 여정' 자체를 자산화.

measure-first 규율

- 시뮬은 항상 seed 고정 → 결과 재현 가능.
- 결과가 나빠도 파라미터를 유리하게 고치지 않는다 (기록만).
→ 데모에만 맞춘 overfitting 함정 회피.
- 새 요인은 '환경'만 어렵게, 추정기 · 방어 로직은 불변으로 측정.

이 규율 덕에 스트레스 · 괴리 수치를 심사에서 그대로 방어할 수 있다.

→ 완성도는 '보기 좋은 숫자'가 아니라 '재현 가능하고 정직하게 측정된 숫자'에서 나온다.

하드웨어 없이 먼저 검증하고 , 정직하게 교정

장애 · 하드웨어 리스크

부품 도착 전 알고리즘을 못 짜면 개발이 지연되고 , 실물에서 처음 검증하면 리스크가 크다 .

해결 · 시물 선 (先) 개발

네트워크 시뮬레이터로 4 대 기능을 먼저 구현 · 검증 . 정상 조건에서 방향 2.12° · 속도 0.10% · 전달률 92.3% · 오탐 0% 확보 .

장애 · 가짜로 좋은 수치

초기엔 바람이 고정 파형이라 시드를 바꿔도 오차가 안 변해 에러바가 가짜로 0 에 가까웠다 .

해결 · 측정 정직화 (#2b)

바람을 시드마다 다른 실현으로 바꿔 진짜 통계로 . 재특성화 결과 : 방향 · 위치는 견고 , 속도는 취약 → ' 방향 신뢰 , 속도 보수적 ' 원칙 확립 .



가장 무섭던 위험 '시계 동기'를 실물 전에 정량화

시물은 모든 노드가 시계 하나를 공유하지만, 실물 ESP32는 저마다 시계가 어긋난다 (드리프트). 우리 추정기는 작은 '시각 차이'로 방향·속도를 뺏기에, 시계 오차가 곧 추정 오차가 될 수 있다 — **시물에선 안 보이는 실물 전용 위험**.

2.13°

방향오차

시계지터 300ms 에서

2.8%

속도오차

300ms 에서

0%

오탐률

통신두절 0.30 까지

12+

권장 노드수

저밀도 속도붕괴 근거

대응 · **painlessMesh** 메시 시각 동기 + 시물 정량화 (#2c)

우리가 고른 라이브러리가 마침 메시 전체 공유 시각 (getNodeTime()) 을 제공 → 드리프트를 근본 완화. 시물로 잔여 지터를 0~300ms 주입해도 방향은 견고 (2.1°). ① 라이브러리 대응 ②시물로 허용치 확보 ③실제값은 실물에서 확인 — 세 겹으로 선점.



'비화재 사망' — 발견부터 해결까지

#2c 에서 **새 축의 한계를 스스로 발굴** — 불이 아닌 죽음 (배터리 · 고장) 을 화선 사망으로 오분류 . ' 찬 구역은 방어된다 ' 는 우리 가설을 측정으로 반증하고 , 원인을 특정해 제거했다 .

진단 (#2e-1 · 측정)

' 위험한 찬 구역은 방어된다 ' 는 가설을 측정으로 반증 — 찬 (COOL) 비화재 누수 86.4%. 원인은 이웃 표본 부족 시 관대 채택 (설계 구멍 , 물리적 한계 아님) 으로 특정 .

해결 (#2e-2)

관대 채택을 멈춰 COOL 누수 86.4%→0%, 방향 오염 제거 (적대적 8 개서 15.7°→2.7°). 남은 통과는 참 화선과 일치하는 무해 HOT 뿐 .
보너스로 기존 오염 breakpoint(0.20) 도 소멸 .

→ 발견→가설→반증→진단→해결의 완결. ' 한계를 아는 ' 것을 넘어 ' 고치는 ' 성숙도. (estimator 수학 불변, 선별 로직만)



고장을 '손실'이 아니라 '데이터'로

비교 대상	그들의 방식	한계	우리 (불사)
온도 스트리밍	살아있는 노드가 온도 상시 전송	초저가 대량 노드엔 상시 전송 부담 (전력 · 대역 폭)	교차검증된 '죽음'을 확정 이진 신호로 → 화선 위치 · 방향 · 속도
Dryad Silvanet	불 오기 전 조기 감지 (내열 , +85℃ 한계)	불 속에선 못 버팀 — 감지 특화 , 전선 추적 아님	타 죽으며 화선을 그림 — 감지 (그들) 와 추적 (우리) 은 상보
stock ESP-MESH	노드 죽으면 경로 우회 (연결성만)	죽음을 손실로만 처리 , 정보로 안 씀	죽음을 1 급 이벤트로 승격 + 도착시각 추정 + 오탐 방어
드론 · 위성	상공 · 우주 광역 개요	연기 · 수목 아래 지표 화선 · 초 단위 변화 못 봄	지표 실측 레이어 + 지상 통신 백본 — 대체 아니라 상보

★ 헤드라인 차별점 오탐 방어 (통신두절 vs 진짜 파괴 구분) = stock 메시가 못 하는 우리만의 계층 .



정상 성능과 '작동 한계선' 을 함께 밝힌다

2.12°

방향오차

정상 조건

0.10%

속도오차

정상 조건 (보수적 해석)

92.3%

전달률

자가치유 포함

0%

오탐률

통신두절 오판

작동 한계선 (operating envelope) — 어디까지 버티는가

축	한계선 / 결과	의미
노드 밀도	12 노드 이상에서 속도 안정 (9 노드서 붕괴)	'12+ 운용' 근거 → 실물 16 + 시물 확장
시계 지터	300ms 까지 방향 2.1° 견고	메시 시각 동기화 사실상 해소
통신두절	측정 전 구간 (0.30) 까지 오탐 0%	관대판정 제거로 초기 breakpoint 소멸
비화 · 비화재	비화 2 개 + 취약 · 비화재 COOL 누수 차단	#2e: COOL 86.4%→0%, 잔여는 무해 HOT

● 실물 정량지표 (재라우팅 실지연 · 추정오차 · 전달률 · 오탐률) 는 부품 도착 후 실측하여 이 자리에 삽입.



누가 쓰고, 어떻게 배포하나

수요자 1 · 진화 지휘부

지표 화선의 실시간 위치 · 방향 · 속도로 '어디에 자원을 먼저 투입할지' 판단 · 자원 배치 정확도 ↑.

수요자 2 · 최전선 대원

'이 구역에 전선 N 분 내 도달' 대피 타이밍 제공 · 대원 안전과 직결 (속도는 안전마진으로 보수적 경보).

배포 · 고위험 구역 사전 배치

상습 발화지, 도시 · 마을 인접 산림 (WUI), 주요 시설 · 문화재 · 등산로에 핀포인트 · 초저가라 촘촘히 깔아도 부담 적음 (사전 배치는 상용 선례로 검증).

기대효과

상공 관측이 놓치는 지표 화선을 지상에서 실시간화 → 자원 배치 정확도와 대원 안전을 동시에 끌어올린다.

저가 소모형의 경제 논리와 확장성

비용 논리

- 노드 개당 수천 원 → 불 속에 들어가는 역할엔 내구성을 '살' 수 없으니 저가 소모형이 유일하게 합리적 .
- 16 노드 실물 $\approx$ 45 만 원 .
- 예방 투자 vs 산불 1 회 피해 (인명 · 산림 · 재산) = 후자가 압도적 .
- 불날 때마다 갈아끼우는 모델이 오히려 경제적 .

발전 가능성

- 같은 ' 고장 - 패턴 - 데이터화 + 자가치유 ' 구조를 실내 화재 · 가스 누출 · 구조물 붕괴 · 홍수로 확장 가능 .
- 온도 상승 궤적 · 곡선 - 허용 시공간 정합 · 다중소스 추정 (#2e) 이 다음 연구 축 .
- 드론 · 위성과 대체가 아니라 상보 레이어로 통합 .

→ 독창성은 특정 응용이 아니라 ' 고장을 데이터로 승격하는 아키텍처 ' 에 있어, 재난 전반으로 이식된다 .

1인 개발자로 전 영역을 단계적으로 수행

단계	기간	내용	상태
① 시뮬 · 알고리즘	~ 완료	네트워크 시뮬 · 도착시각 추정 · 스트레스 · 괴리 정량화 (#2c)	✅ 완료
② 실물 포팅	부품 후 ~1 주	16 노드 플래시 · 메시 / 임종신호 실제 RF 검증 · 통합	진행 예정
③ 데모 · 측정	이후	열원 트리거 데모 · 재라우팅 / 추정오차 / 전달률 실측	🟡 실측
④ 문서 · 영상	~ 제출 (9/4)	보고서 PPT 완성 · 시연영상 · GitHub Public	진행 중

1인 업무 분장 (영역별)

펌웨어 (ESP32·painlessMesh) · 알고리즘 / 게이트웨이 (Python 추정기) · 하드웨어 (16 노드 조립 · 데모) · 문서 / 발표 (보고서 · 영상) 를 단독으로 단계적 수행. 시뮬 선개발로 리스크를 앞당겨 해소하고, measure-first 로 범위를 통제 — 1인 개발의 범위 관리 · 완결 역량을 보인다.

